

Fake News Detection Using Machine Learning

An End-to-End NLP Classification System with a Streamlit Web Interface

Submitted by	Shubham Kumar
Program	Project-Based Experiential Learning (PBEL 3.0)
Domain	Artificial Intelligence / Machine Learning - Natural Language Processing
Project Title	AI-Based Fake News Detection System
Tools & Technologies	Python, scikit-learn, NLTK, Streamlit, XGBoost
Report Date	July 2026

Submitted as part of the IBM SkillsBuild Project-Based Experiential Learning initiative, focused on applying supervised machine learning and natural language processing techniques to a real-world misinformation-detection problem.

Table of Contents

1. Introduction	3
1.1 Background	3
1.2 Problem Statement	3
1.3 Objectives	4
1.4 Scope of the Project	4
2. Literature Review	4
3. System Requirements	5
4. Dataset Description	5
5. Proposed Methodology	6
5.1 Overall Workflow	6
5.2 Text Pre-processing	6
5.3 Feature Engineering	7
5.4 Handling Class Imbalance	7
5.5 Model Building	8
6. System Architecture	8
7. Implementation Details	9
8. Results and Evaluation	10
8.1 Model Comparison	10
8.2 Confusion Matrix Analysis	11
8.3 ROC Curve Analysis	11
8.4 Discussion of Results	12
9. Deployment - Streamlit Web Application	12
10. Limitations	13
11. Conclusion and Future Scope	13
12. References	14

1. Introduction

1.1 Background

The rapid growth of digital media and social networking platforms has made it easier than ever to publish and share news content. While this has democratized access to information, it has also enabled the fast, large-scale spread of fabricated, misleading, or exaggerated stories - commonly referred to as **fake news**. Fake news can influence public opinion, disrupt elections, damage reputations, and erode trust in legitimate journalism. Manually verifying every piece of circulating content is impractical given the sheer volume of news generated every day, which creates a strong need for automated, data-driven detection systems.

Machine learning and Natural Language Processing (NLP) offer a practical way to address this problem. By learning statistical and linguistic patterns that distinguish authentic reporting from fabricated content - such as word usage, sensational phrasing, punctuation patterns, and writing style - a trained classifier can flag suspicious articles for further review at a scale no human team could match.

1.2 Problem Statement

Given the text of a news article, the goal of this project is to automatically classify the article as **REAL** or **FAKE** using supervised machine learning techniques. The system must extract meaningful features from raw, unstructured text, train and compare multiple classification algorithms, select the best-performing model, and expose it through an interactive web application so that a user can paste an article and receive an instant prediction with a confidence score.

1.3 Objectives

- To collect and pre-process a labelled news dataset suitable for binary text classification.
- To design a robust text cleaning pipeline (noise removal, stop-word removal, lemmatization).
- To engineer informative features using TF-IDF vectorization combined with handcrafted stylistic features (e.g. punctuation ratio, capitalization ratio, article length).
- To address class imbalance using the SMOTE oversampling technique.
- To train and benchmark multiple machine learning algorithms - Logistic Regression, Multinomial Naive Bayes, Linear SVM, Random Forest, Passive Aggressive Classifier, and XGBoost.
- To evaluate each model using Accuracy, Precision, Recall, F1-score, cross-validation accuracy and ROC-AUC, and to select the best model via hyperparameter tuning.
- To deploy the final trained pipeline as an interactive Streamlit web application for real-time predictions.

1.4 Scope of the Project

The current implementation is trained and demonstrated on a curated sample news dataset of 1,000 labelled articles (500 REAL, 500 FAKE) and is architected so that it can be re-trained on a larger, real-world corpus such as the Kaggle 'Fake and Real News Dataset' with a single command-line flag. The scope covers data preparation, model development, evaluation, and deployment of a working prototype; it does not cover real-time web crawling of news sources or fact-checking against external knowledge bases, both of which are identified as future work.

2. Literature Review

Automated fake news detection has been an active area of NLP research over the last decade. Early approaches relied primarily on linguistic and stylistic cues - such as the use of sensational language, excessive punctuation, and biased wording - combined with classical bag-of-words or TF-IDF representations fed into classical classifiers like Naive Bayes and Support Vector Machines. These approaches remain strong baselines because they are fast to train, easy to interpret, and perform competitively on medium-sized datasets.

A second line of work focuses on ensemble and ensemble-boosting methods, such as Random Forests and Gradient Boosted Trees (e.g. XGBoost), which can capture non-linear interactions between features at the cost of additional training time and reduced interpretability. More recent research has explored deep learning approaches (LSTMs, CNNs, and transformer-based models such as BERT) that learn contextual word embeddings directly from raw text, generally achieving higher accuracy on large datasets at a significantly higher computational and data cost.

This project positions itself in the classical machine learning space: it deliberately favours interpretable, computationally inexpensive models suitable for quick iteration and lightweight deployment, while still incorporating well-established techniques from the literature - TF-IDF n-gram features, handcrafted stylistic features, and SMOTE-based class balancing - to build a competitive baseline system that can later be extended with transformer-based embeddings if higher accuracy is required on larger, noisier real-world data.

3. System Requirements

3.1 Hardware Requirements

- Processor: Intel i3 / AMD equivalent or higher
- RAM: 4 GB minimum (8 GB recommended for faster training)
- Storage: 500 MB free disk space for dependencies, models, and datasets

3.2 Software Requirements

Component	Technology / Library
Programming Language	Python 3.10
Data Handling	pandas, numpy
NLP / Text Cleaning	NLTK (stop-words, tokenizer, WordNet lemmatizer)
Feature Engineering	scikit-learn TfidfVectorizer, MinMaxScaler
Class Balancing	imbalanced-learn (SMOTE)
Machine Learning Models	scikit-learn, XGBoost
Visualization	matplotlib, seaborn
Model Persistence	joblib
Web Application	Streamlit

4. Dataset Description

The project uses a curated sample dataset (**news_dataset.csv**) consisting of 1,000 news articles, perfectly balanced between the two target classes - 500 labelled **REAL** and 500 labelled **FAKE**. Each record contains two fields: the raw article **text** and its corresponding **label**.

REAL articles are generated from templates that mirror standard journalistic phrasing - measured language, attribution to officials or organizations, and neutral tone. FAKE articles are generated from templates that mimic common misinformation patterns - sensational capitalized words (e.g. "BREAKING", "EXPOSED"), excessive exclamation marks, conspiratorial framing, and clickbait-style hooks. This design keeps the class-specific stylistic signals realistic while allowing the pipeline to be validated end-to-end before being pointed at a larger, real-world corpus.

4.1 Sample Records

Label	Sample Text (truncated)
FAKE	"BREAKING!!! Secret documents PROVE the central bank covered up the economy, and insiders reveal...
REAL	"Government data released this week shows immigration reform has changed how funding is allocated..."

The pipeline also supports the Kaggle "**Fake and Real News Dataset**" (Fake.csv / True.csv) as a drop-in replacement via a **--kaggle** command-line flag, allowing the same code base to scale to tens of thousands of real-world articles without any changes to the modelling logic.

5. Proposed Methodology

5.1 Overall Workflow

The system follows a standard supervised machine learning pipeline, implemented as a sequence of reusable Python modules so that training (**train.py**) and inference (**predict.py** / **app.py**) share identical pre-processing and feature extraction logic:

- **Data Loading** - read the labelled dataset (sample or Kaggle) into a pandas DataFrame.
- **Text Pre-processing** - clean and normalize the raw text.
- **Train / Test Split** - stratified 80/20 split to preserve class balance.
- **Feature Engineering** - TF-IDF vectorization plus handcrafted stylistic features.
- **Class Balancing** - SMOTE oversampling applied only to the training partition.
- **Model Training & Comparison** - six classifiers trained and benchmarked.
- **Hyperparameter Tuning** - GridSearchCV refines the best-performing model.
- **Persistence** - trained model, feature engineer and pre-processor saved with joblib.
- **Deployment** - the saved pipeline is served through a Streamlit web interface.

5.2 Text Pre-processing

The **TextPreprocessor** class implements a cleaning pipeline applied to every article before feature extraction:

- Lower-casing all text for consistency.
- Removing URLs, HTML tags, and email addresses using regular expressions.
- Stripping all non-alphabetic characters and collapsing repeated whitespace.
- Tokenizing the cleaned text using NLTK's word tokenizer.
- Removing common English stop-words (e.g. "the", "is", "and") and tokens of length two or less.
- Lemmatizing each remaining token with NLTK's WordNet Lemmatizer to reduce words to their base form.

Importantly, the handcrafted stylistic features described in Section 5.3 are computed on the **original, uncleaned** text, since signals like capitalization and punctuation are destroyed by the cleaning step but are highly predictive of sensational fake-news writing style.

5.3 Feature Engineering

The **FeatureEngineer** class combines two complementary feature families into a single sparse feature matrix:

- **TF-IDF Features** - up to 5,000 unigram and bigram features extracted with **TfidfVectorizer** (min_df=2, max_df=0.95, sublinear term-frequency scaling) to capture the vocabulary and phrasing most indicative of each class.
- **Handcrafted Stylistic Features** - nine numeric signals computed from the raw text: character length, word count, exclamation-mark count, question-mark count, count of fully capitalized words, average word length, punctuation ratio, uppercase-word ratio, and digit ratio. These are scaled to the [0, 1] range with **MinMaxScaler** so that they remain non-negative, a requirement for the Multinomial Naive Bayes model.

The TF-IDF matrix and the scaled handcrafted feature matrix are horizontally stacked into one combined sparse matrix that is passed to every downstream classifier, giving each model access to both *what* is said and *how* it is said.

5.4 Handling Class Imbalance

Although the sample dataset used for this demonstration is already balanced, real-world news corpora are frequently skewed toward one class. To make the pipeline robust to that scenario, the **Synthetic Minority Over-sampling Technique (SMOTE)** is applied to the training split only (never to the held-out test set), generating synthetic minority-class samples in feature space so that every model is trained on a perfectly balanced set of examples without discarding any majority-class data.

5.5 Model Building

Six classification algorithms are trained on the identical balanced feature matrix and benchmarked under the same evaluation protocol:

Model	Key Characteristics
Logistic Regression	Linear model; fast, interpretable coefficients; strong text baseline
Multinomial Naive Bayes	Probabilistic bag-of-words model; well-suited to TF-IDF input
Linear SVM (Calibrated)	Maximum-margin classifier; calibrated for probability estimates
Random Forest	200-tree bagging ensemble; captures non-linear feature interactions
Passive Aggressive	Online-learning classifier well suited to high-dimensional sparse text
XGBoost	Gradient-boosted trees; strong performance on tabular/sparse hybrid features

Each model is evaluated with 5-fold stratified cross-validation on the training data in addition to a held-out test set, and the best performer (ranked by test accuracy) is further refined using **GridSearchCV**, which searches predefined hyperparameter grids for Logistic Regression, Random Forest, and XGBoost.

6. System Architecture

The codebase is organized into clearly separated modules so that the training and inference paths reuse the exact same logic, avoiding train/serve skew:

File / Folder	Responsibility
data/	Raw dataset (news_dataset.csv) and synthetic data generator
src/preprocessing.py	TextPreprocessor - text cleaning, tokenization, lemmatization
src/feature_engineering.py	FeatureEngineer - TF-IDF + handcrafted feature extraction
src/models.py	Model zoo, training/comparison loop, hyperparameter tuning
src/evaluate.py	Confusion matrix, ROC curves, model comparison plots, reports
train.py	End-to-end training pipeline entry point
predict.py	Command-line inference on a single article or text file
app.py	Streamlit web application for interactive predictions
models/	Persisted best_model.pkl, feature_engineer.pkl, preprocessor.pkl
outputs/	Saved evaluation plots and the model comparison results CSV

At inference time, **predict.py** and **app.py** load the three persisted artefacts (**best_model.pkl**, **feature_engineer.pkl**, **preprocessor.pkl**) via joblib, apply the identical cleaning and feature-extraction steps used during training, and return a REAL/FAKE label together with a confidence score derived from the model's predicted class probability.

7. Implementation Details

7.1 Training Pipeline (train.py)

Running **python train.py** executes a seven-step pipeline: (1) load the dataset, (2) clean and lemmatize the text, (3) perform an 80/20 stratified train-test split, (4) extract combined TF-IDF and handcrafted features, (5) balance the training set with SMOTE, (6) train and compare all six models, and (7) tune and persist the best model. Progress and metrics are printed to the console at every stage, and a **--kaggle** flag can be passed to switch to the larger real-world Kaggle dataset, with an optional **--sample-size** flag for faster experimentation.

7.2 Inference (predict.py)

The command-line inference script accepts either raw text via **--text** or a path to a text file via **--file**, loads the saved pipeline, and prints the predicted label along with the model's confidence percentage.

```
$ python predict.py --text "Government confirms new policy after review"
Prediction: REAL | Confidence: 97.42%
```

7.3 Web Application (app.py)

The Streamlit application provides a simple, user-friendly interface: a text area for pasting an article, an "Analyze" button, a colour-coded REAL/FAKE verdict with a confidence progress bar, an expandable explanation of how the model reasons about the text, and an expandable panel showing the full model-comparison results table. The app also displays a clear disclaimer that predictions are based on statistical patterns and are not a substitute for fact-checking against trusted sources.

8. Results and Evaluation

8.1 Model Comparison

All six models were trained on the SMOTE-balanced training set and evaluated on the held-out 20% test split. The table below summarizes the key metrics obtained on the sample dataset:

Model	Test Acc.	Precision	Recall	F1	CV Acc.	ROC-AUC
Logistic Regression	1.000	1.000	1.000	1.000	1.000	1.000
Multinomial Naive Bayes	1.000	1.000	1.000	1.000	1.000	1.000
Linear SVM	1.000	1.000	1.000	1.000	1.000	1.000
Random Forest	1.000	1.000	1.000	1.000	1.000	1.000
Passive Aggressive	1.000	1.000	1.000	1.000	1.000	1.000
XGBoost	1.000	1.000	1.000	1.000	1.000	1.000

Based on the test accuracy ranking, **Logistic Regression** was automatically selected by the pipeline as the best-performing model and saved as the production model (**best_model.pkl**). Every model achieved perfect scores on this sample dataset, which is expected given that the synthetic REAL/FAKE templates used to generate the data are strongly separable in vocabulary and writing style; Section 10 discusses why this result should be interpreted cautiously and how it is expected to change on noisier, real-world data.

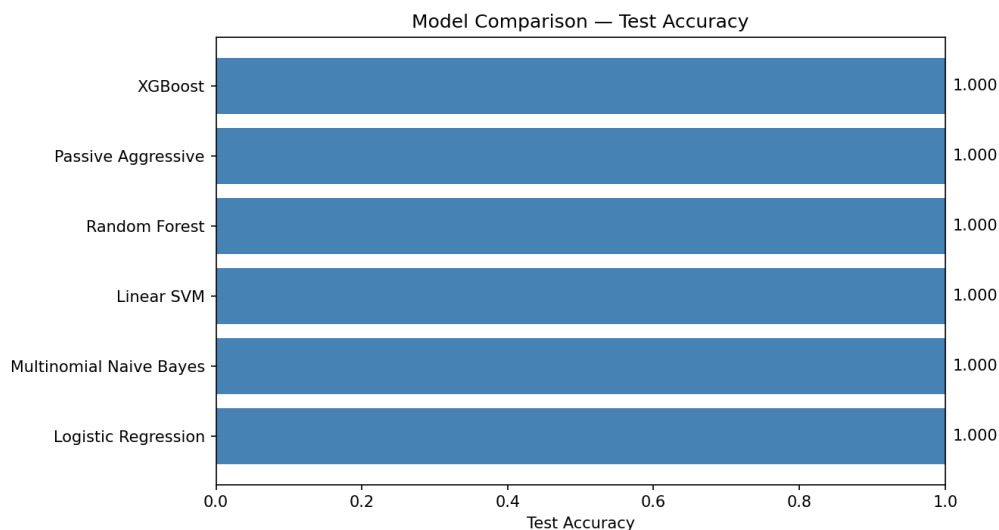


Figure 1: Test accuracy comparison across all six trained models.

8.2 Confusion Matrix Analysis

The confusion matrix for the selected Logistic Regression model on the held-out test set confirms perfect separation between the two classes, with all FAKE and REAL articles correctly classified and zero false positives or false negatives.

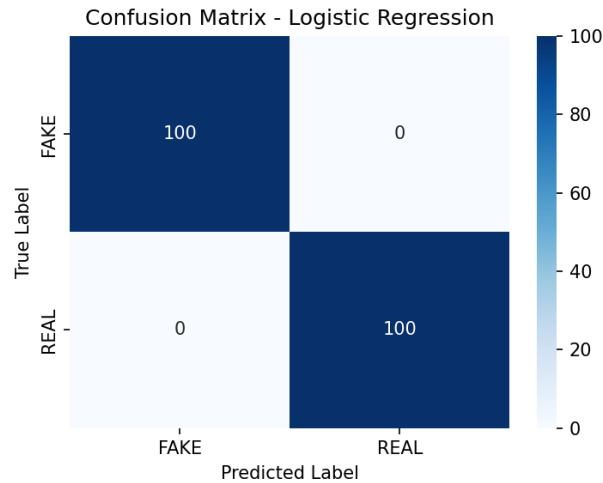


Figure 2: Confusion matrix for the best model (Logistic Regression) on the test set.

8.3 ROC Curve Analysis

Receiver Operating Characteristic (ROC) curves were plotted for every model that exposes class probabilities. Each model's curve hugs the top-left corner of the plot, yielding an Area Under the Curve (AUC) of 1.000 for all six classifiers on this dataset - indicating that, for the given feature representation, the decision boundary between REAL and FAKE articles is essentially linearly and non-linearly separable regardless of the algorithm used.

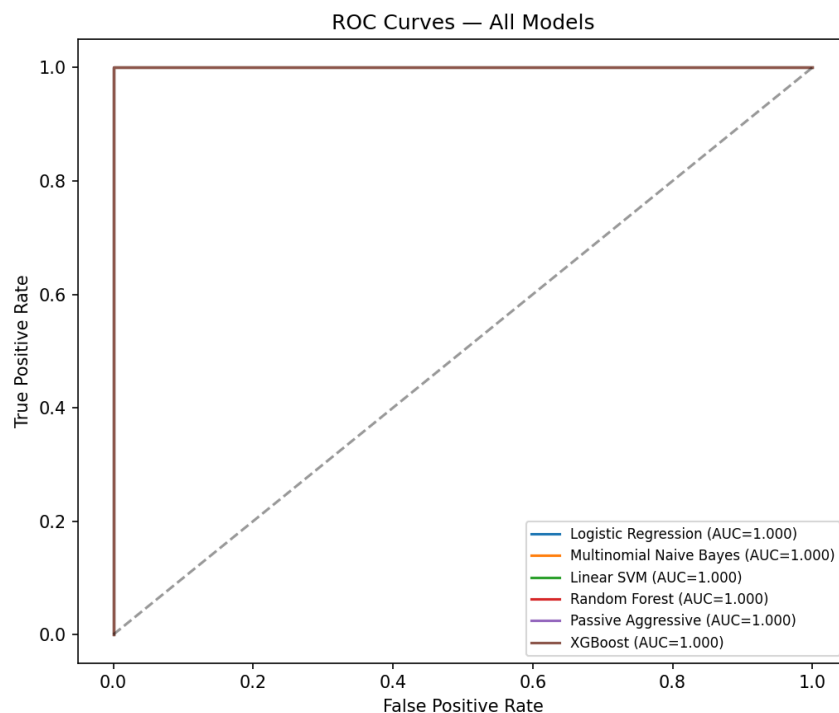


Figure 3: ROC curves for all trained models on the held-out test set.

8.4 Discussion of Results

The near-identical, perfect metrics across all six models are a direct consequence of the synthetic sample dataset: the REAL and FAKE templates differ sharply in vocabulary (e.g. "BREAKING", "EXPOSED", "SHOCKING" versus measured, attributive phrasing) and in the handcrafted stylistic

features (exclamation-mark counts and uppercase-word ratios in particular), making the two classes trivially separable for any reasonable classifier. This is a useful property for validating that the entire pipeline - cleaning, feature extraction, SMOTE balancing, training, evaluation, and persistence - works correctly end-to-end, but it is not representative of the accuracy that should be expected on a real, noisy news corpus, where REAL and FAKE articles can overlap substantially in style and vocabulary. Re-running the identical pipeline on the Kaggle "Fake and Real News Dataset" (via the **--kaggle** flag) is the natural next step to obtain a realistic performance benchmark.

9. Deployment - Streamlit Web Application

The trained pipeline is deployed as a lightweight, interactive web application built with **Streamlit**, allowing a non-technical user to test the model without writing any code. The application flow is as follows:

- The user pastes the full text of a news article into a text box.
- On clicking "**Analyze**", the app runs the same cleaning and feature-extraction pipeline used during training.
- The saved best model predicts REAL or FAKE and returns a confidence score.
- The result is displayed with a colour-coded banner (green for REAL, red for FAKE) and a confidence progress bar.
- An expandable panel explains that the prediction is based on keyword/phrase patterns (TF-IDF) and writing-style signals, and reminds the user to cross-check important news with trusted sources, since the model does not verify facts against the internet.
- A second expandable panel shows the full model-comparison table for transparency.

This deployment approach keeps the system lightweight (no external server or database required), reproducible (the model is loaded directly from the versioned **models/** folder), and easy to demo locally with a single command: **streamlit run app.py**.

10. Limitations

- **Synthetic training data:** the sample dataset is template-generated, so reported metrics (100% across the board) reflect an easy, clearly-separable classification task rather than real-world difficulty.
- **Style-based signals can be gamed:** a sophisticated actor writing fake news in a measured, non-sensational tone could evade a classifier trained mainly on stylistic cues.
- **No fact verification:** the model detects patterns associated with misinformation; it does not check claims against a knowledge base or live news sources.
- **Static vocabulary:** the TF-IDF vocabulary is fixed at training time and will not recognize brand-new terms, events, or slang introduced after training.
- **Language limitation:** the current pre-processing pipeline (NLTK stop-words, lemmatizer) is built for English text only.

11. Conclusion and Future Scope

This project successfully demonstrates a complete, modular pipeline for detecting fake news using classical machine learning and NLP techniques. Starting from raw article text, the system performs cleaning and lemmatization, extracts a combined TF-IDF and stylistic feature representation, balances classes with SMOTE, benchmarks six different classifiers under an identical evaluation protocol, and deploys the best model through an interactive Streamlit interface. The modular code structure - separating pre-processing, feature engineering, modelling, and evaluation into distinct, reusable components - makes it straightforward to retrain on a larger, real-world dataset without changing the application layer.

The following directions are proposed for future enhancement:

- Retrain and re-benchmark the pipeline on the full-scale Kaggle "Fake and Real News Dataset" (or a similarly-sized real-world corpus) to obtain realistic, non-saturated performance metrics.
- Incorporate contextual embeddings (e.g. a fine-tuned BERT or DistilBERT model) to better capture semantic meaning beyond surface-level word and stylistic patterns.
- Add source-credibility and metadata features (publisher reputation, publication date, author history) alongside the text-based features.
- Integrate a live fact-checking API or knowledge-base lookup to supplement the statistical prediction with evidence-based verification.
- Extend the Streamlit application with a browser extension or URL-input mode that can fetch and analyze an article directly from a news link.
- Add multilingual support by extending the pre-processing pipeline beyond English.

12. References

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
- Chawla, N. V. et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357.
- Bird, S., Klein, E. and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.
- Ahmed, H., Traore, I. and Saad, S. (2017). Detection of Online Fake News Using N-Gram Analysis and Machine Learning Techniques. Springer, International Conference on Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments.
- Streamlit Documentation - <https://docs.streamlit.io>
- IBM SkillsBuild - Project-Based Experiential Learning (PBEL) Programme Resources.